API Performance Optimization

How I achieved

40% FASTER APIs

Before	After
800ms	480ms

Real production techniques →

1 Query Optimization

Query Time Reduced

2.3s → 0.9s

THE PROBLEM

N+1 queries and missing indexes were killing performance. Database was the bottleneck.

THE SOLUTION

Use JOIN FETCH
Eliminated N+1 queries in one shot

Add Strategic Indexes
60% faster lookups on frequently queried columns

Implement Pagination
Load only what you need, when you need it

SELECT Specific Fields
Dont fetch entire objects if you need 2 fields

2 Redis Caching

Database Load Reduced

70% Less Calls

THE PROBLEM

Repeatedly querying database for same data. Every request hit the DB.

THE SOLUTION

Cache-Aside Pattern
Check cache first, then DB if miss

TTL = 15 minutes
Auto-expire stale data

Cache Warming
Pre-load frequently accessed data on startup

Smart Invalidation
Update cache when data changes

3 Connection Pooling

DB Operations Faster

25% Boost

THE PROBLEM

Creating new DB connections for every request. Huge overhead.

THE SOLUTION

HikariCP Configuration
Fastest connection pool for Java

Pool Size = CPU × 2
Optimal for most applications

Connection Timeout = 30s
Fail fast, dont wait forever

Max Lifetime = 30min
Rotate connections to avoid stale ones

4 Smart Pagination

Response Size

500KB → 15KB

THE PROBLEM

Loading 10,000 records when user needs only 20. Slow and wasteful.

THE SOLUTION

Page Size = 20
Perfect balance between UX and performance

Use Spring Pageable
Built-in support, super easy

```
@GetMapping("/api/transactions")
public Page<Transaction> getAll(
    Pageable pageable) {
    return repo.findAll(pageable);
}
```

5 Async Processing

User Response Time

Instant ⚡

THE PROBLEM

Email sending and PDF generation blocking user request. 5s wait.

THE SOLUTION

@Async Annotation
Offload heavy tasks to background threads

Email Notifications
Send after response, user doesnt wait

File Processing
Upload now, process later, notify when done

External API Calls
Dont block user for third-party delays

6 Monitor Everything

Bug Detection Speed

50% Faster

THE PROBLEM

No visibility into what is slow. Flying blind. Cant fix what you cant see.

THE SOLUTION

Log4j Logging
Track every request, response, and error

Spring Actuator
Built-in health checks and metrics

Slow Query Log
Find problematic database queries

APM Tools
Real-time performance monitoring

Key Takeaways

6 Proven Techniques

01	**Query Optimization** 61% faster
02	**Redis Caching** 70% less DB calls
03	**Connection Pooling** 25% improvement
04	**Smart Pagination** 97% less data
05	**Async Processing** Instant response
06	**Monitoring** 50% faster debug

40% FASTER